

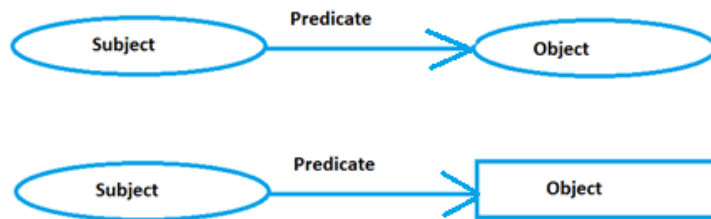
Введение в RDF

Апанович З.В

IIS SBRAS, NSU

Модель данных RDF

Модель данных RDF представляет информацию как ориентированные графы, с помеченными вершинами и ребрами



Модель данных RDF создана для интегрированного представления информации,

- происходящей из множественных источников,
- гетерогенно структурированной,
- и представленной при помощи разных схем.

IRIs

- IRI – это глобальные идентификаторы, так что другие люди могут повторно использовать эти IRI , чтобы идентифицировать одно и то же.
- Например, следующее IRI используется многими людьми как свойство RDF, чтобы заявить об отношении знакомства между людьми:
- <http://xmlns.com/foaf/0.1/knows>
- RDF является агностиком о том, что представляет IRI. Тем не менее, IRI может быть дано значение конкретными словарей или конвенций.
- Например, DBpedia использует IRI <http://dbpedia.org/resource/Name> для обозначения сущности, описанной в соответствующей статье Википедии.

RDF Model

Высказывание на естественном языке:

<http://www.example.org/index.html> has
a **creator** whose value is **John Smith**

Structure

Ресурс (**subject**)

<http://www.example.org/index.html>

Свойство (**predicate**) “creator”

здесь: literal

Значение (**object**)

“John Smith”

здесь: literal



Соответствующий граф



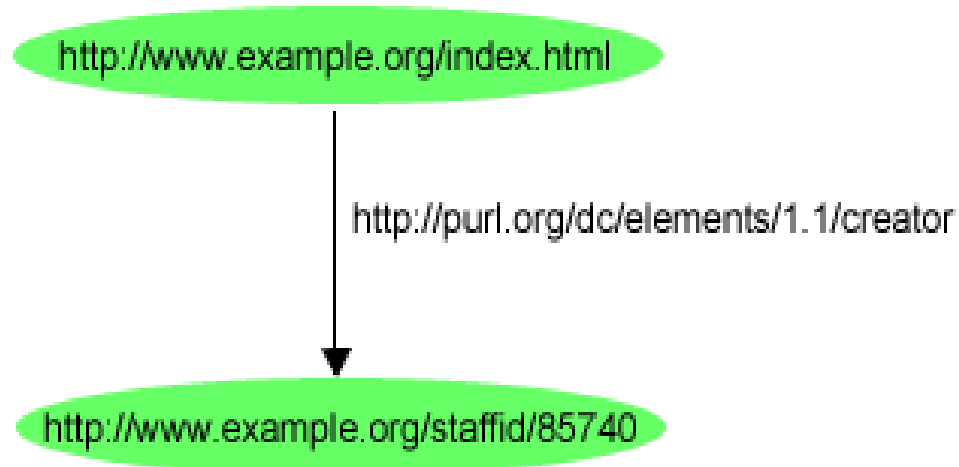
В графовой нотации
высказывание представляется:

- вершиной для **subject**,
- вершиной для **object**,
- ориентированным ребром для **predicate**, направленным от вершины-subject к вершине object.

RDF Model

**Какой вариант
лучше???**

<http://www.example.org/index.html>
<http://purl.org/dc/elements/1.1/creator>
<http://www.example.org/staffid/85740> .

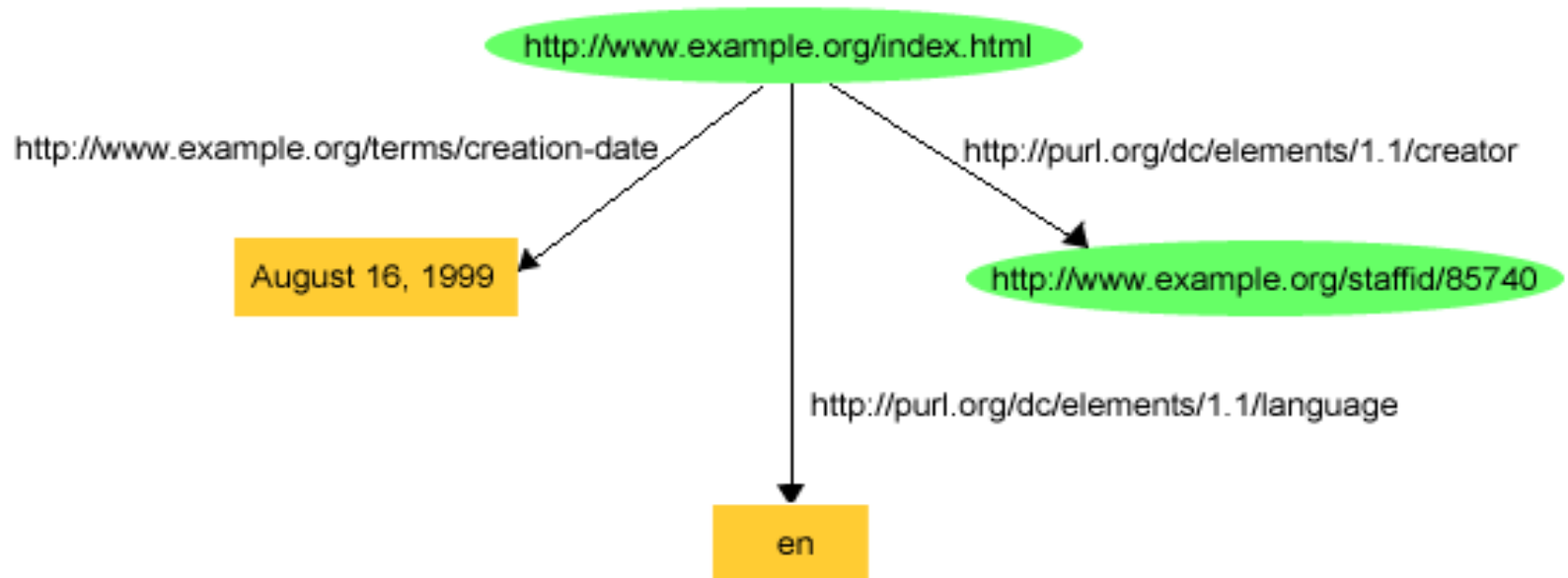


RDF Model

<<http://www.example.org/index.html>>
<<http://purl.org/dc/elements/1.1/creator>>
<<http://www.example.org/staffid/85740>> .

<<http://www.example.org/index.html>>
<<http://www.example.org/terms/creation-date>> "August 16, 1999" .

<<http://www.example.org/index.html>>
<<http://purl.org/dc/elements/1.1/language>> "en" .



RDF в виде графа (ов)

Вершины:

- Ресурсы представленные своим URIs
- Ресурсы без имени (**Blank Nodes**)
- Литералы, представленные в виде Strings

Ориентированные ребра:

- Представлены URIs

Использование сокращений

URI состоит из префикса и индивидуального локального имени:

<http://www.example.org/index.html>

<http://purl.org/dc/elements/1.1/creator>

<http://www.example.org/staffid/85740> .

Qname:

ex:index.html

dc:creator

exstaff:85740

Полные высказывания и сокращения

<http://www.example.org/index.html>
<http://purl.org/dc/elements/1.1/creator>
<http://www.example.org/staffid/85740> .

<http://www.example.org/index.html>
<http://www.example.org/terms/creation-date> "August 16, 1999" .

<http://www.example.org/index.html>
<http://purl.org/dc/elements/1.1/language> "en" .

Сокращения:

ex:index.html dc:creator exstaff:85740 .

ex:index.html exterms:creation-date "August 16, 1999" .

ex:index.html dc:language "en" .

Словари

- Поскольку RDF использует IRIs для именования сущностей в высказываниях, RDF говорит про множество IRIs (предназначенных для определенных целей) как про словари (*vocabulary*).
- Часто IRIs в таких словарях организованы таким образом, что они могут быть представлены как множество имен, использующих общий префикс.
- Это значит, что общее пространство имен IRIs будет использоваться для всех терминов словаря, обычно под контролем того, кто определяет словарь.
- IRIs, содержащиеся в словаре, образуются присоединением индивидуальных локальных имен к концу общего префикса. Это дает множество URIs с общим префиксом.

Словари

RDF использует этот же подход для определения своего собственного словаря терминов со специальным смыслом в RDF.

Все IRI в этом RDF словаре начинаются с

<http://www.w3.org/1999/02/22-rdf-syntax-ns#>,

который по соглашению связан с префиксом [rdf:](#)

Словари

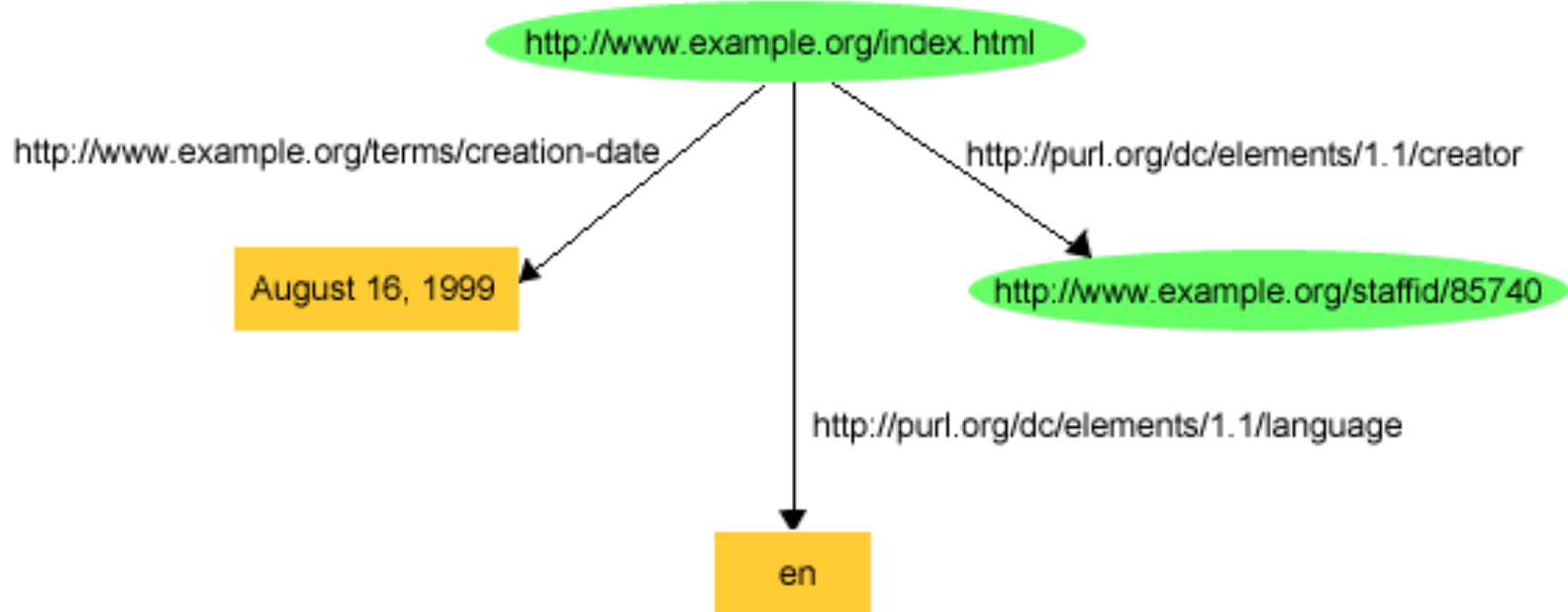
Термин **словарь** будет использоваться при обращении к набору IRI, определенных для каких-либо целей.

Термин **пространство имен** (*namespace*) будет использоваться только при обсуждении синтаксической концепции пространства имен XML.

IRIs из различных словарей можно свободно смешивать в графах RDF.

Например,

URIrefs из словарей [exterms:](#), [exstaff:](#) и [dc:](#).

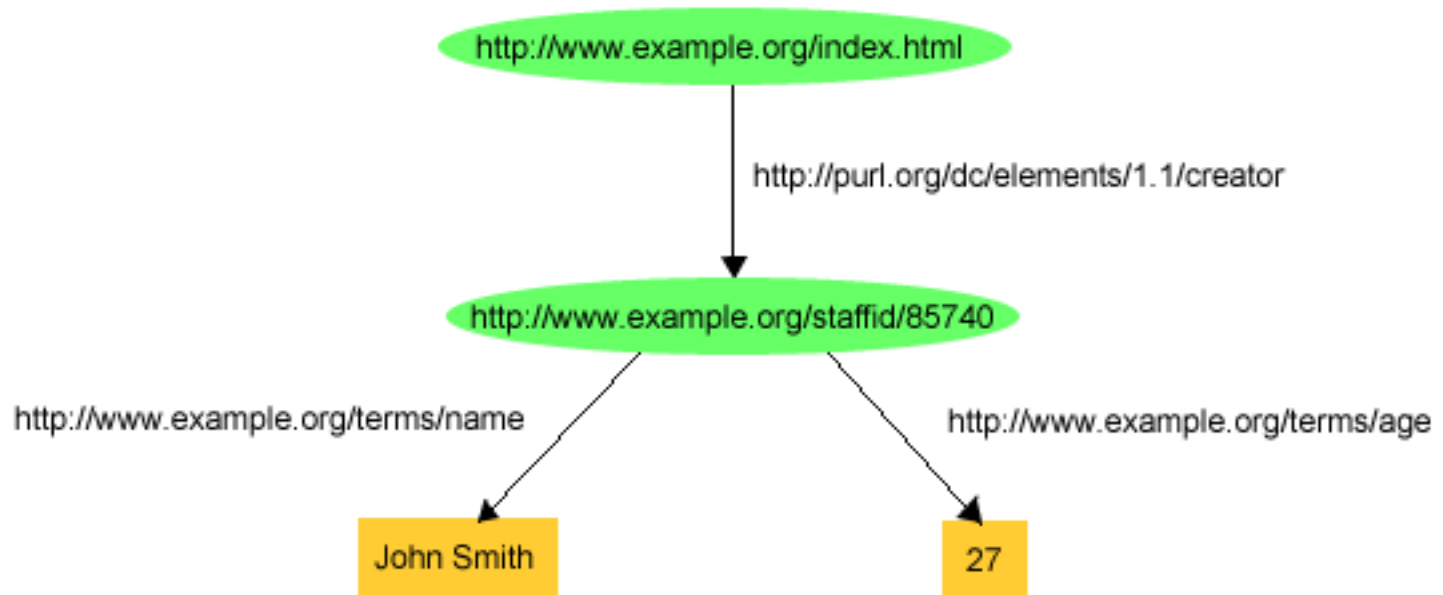


Нет ограничений на то, сколько высказываний, использующих данное IRI в качестве **предиката**, может появиться в графе

Если ресурс [ex:index.html](#) был создан совместными усилиями нескольких сотрудников помимо John Smith, то могут появиться такие высказывания в example.org :

- [ex:index.html](#) **dc:creator** [exstaff:85740](#) .
- [ex:index.html](#) **dc:creator** [exstaff:27354](#) .
- [ex:index.html](#) **dc:creator** [exstaff:00816](#) .

Польза от использования IRIs



- 1) Идентификацию субъекта высказывания можно сделать более точной. В качестве создателя страницы можно указать не цепочку литер "John Smith", которым может быть один из нескольких тысяч Джонов Смитов, но вполне определенного Джона Смита, связанного с конкретным IRI
- 2) Так как свойства являются ресурсами, о них может быть записана дополнительная информация (например, описание того, что example.org подразумевает под «name»), просто добавив дополнительные высказывания RDF к URI свойства в качестве субъекта.

Использование IRIs для идентификации свойств (предикатов)

- 1) Это отличает свойства, которые может использовать один человек, от свойств, которые может использовать кто-то другой, **в противном случае они могли бы быть** идентифицированы одной и той же строкой символов.
- 2) Свойства сами можно рассматривать как ресурсы.
- 3) Поддерживает развитие и использование общих словарей в Интернете, так как люди могут найти и начать использовать словари, которые уже используются другими для описания сущностей, отражая общее понимание этих понятий.

Использование IRIs для идентификации properties (договоренности о том как понимать)

Например: `ex:index.html` `dc:creator` `exstaff:85740` .

Предикат `dc:creator`, является однозначной ссылкой на атрибут "creator" в [Dublin Core metadata attribute set](#).

Создатель из этой тройки сказал, что отношение между веб-страницей (<http://www.example.org/index.html>) и создателем страницы (<http://www.example.org/staffid/85740>) определены в точности отношением <http://purl.org/dc/elements/1.1/creator>.

Другой человек, знакомый со словарем Dublin Core, или тот кто найдет, что означает `dc:creator` (скажем поиском его определения в Интернете) будет знать, что подразумевается под этим отношением .

Кроме того, на основе этого понимания люди могут писать программы, которые ведут себя в соответствии с этим смыслом при обработке тройки , содержащей предикат `dc:creator`.

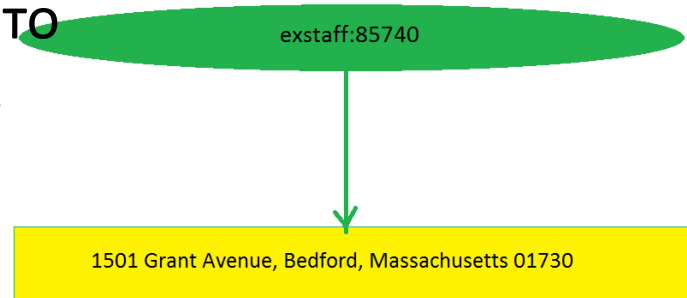
Идентификаторы пустых узлов (Blank node)

- Идентификаторы пустых узлов – это локальные идентификаторы, которые используются в некоторых конкретных синтаксисах RDF или реализациях RDF хранилища.
- Они всегда имеют локальную область действия в файле или RDF хранилище, и не являются устойчивыми или переносимыми идентификаторами пустых узлов.

Структурированные значения свойств и пустые узлы

В случае личной информации

Джона Смита, предположим, что адрес Джона был описан. Весь адрес может быть записан как один литерал :



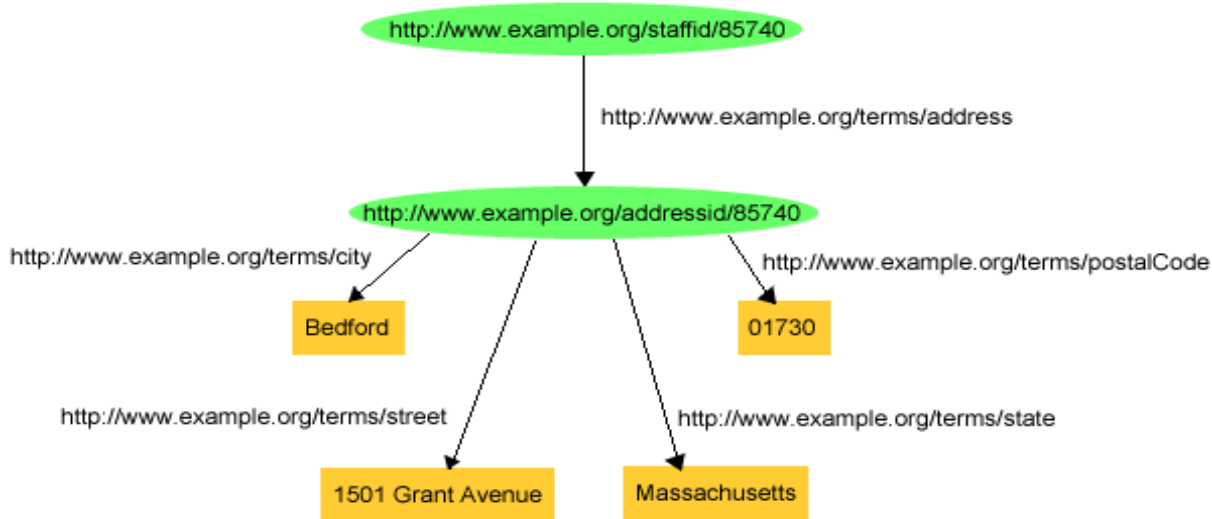
exstaff:85740 extermis:address

"1501 Grant Avenue, Bedford,
Massachusetts 01730" .

Однако предположим, что адрес Джона нужно записать в виде **структуры**, состоящей из отдельных улицы, города, штата и почтовых кодовых значений?

Как это сделать в RDF??

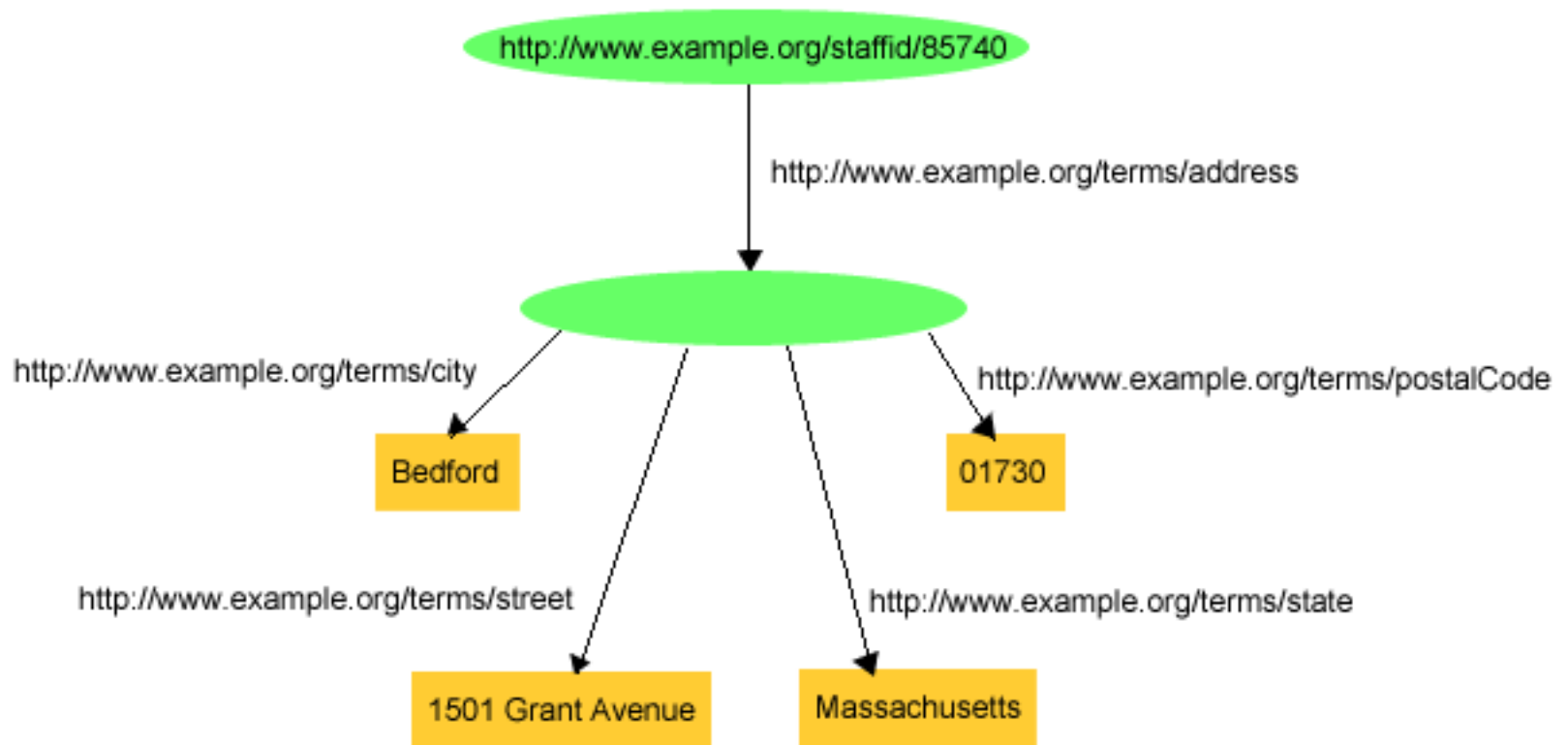
Структурированные значения свойств и пустые узлы



Triples:

- `exstaff:85740` `exterms:address` `exaddressid:85740` .
- `exaddressid:85740` `exterms:street` `"1501 Grant Avenue"` .
- `exaddressid:85740` `exterms:city` `"Bedford"` .
- `exaddressid:85740` `exterms:state` `"Massachusetts"` .
- `exaddressid:85740` `exterms:postalCode` `"01730"` .

Пустой узел



Пустой узел

Тем не менее, некоторая форма явного идентификатора необходима для этого узла, чтобы представить этот граф в виде троек.

```
exstaff:85740  exterms:address      ??? .  
???           exterms:street       "1501 Grant Avenue" .  
???           exterms:city         "Bedford" .  
???           exterms:state        "Massachusetts" .  
???           exterms:postalCode   "01730" .
```

??? Изображает что-то, что указывает на присутствие пустой вершины.

Пустой узел

Поскольку сложный граф может содержать более одного пустого узла, также должен быть способ различать эти различные пустые узлы в триплетном представлении графа.

В результате тройки используют идентификаторы пустых узлов, имеющие форму `_:ИМЯ`, чтобы указать на факт наличия пустых узлов.

```
exstaff:85740  exterm:s:address    _:johnaddress .
_:johnaddress exterm:s:street  "1501 Grant Avenue" .
_:johnaddress exterm:s:city    "Bedford" .
_:johnaddress exterm:s:state   "Massachusetts" .
_:johnaddress exterm:s:postalCode "01730" .
```

Пустой узел

- 1) В триплетном представлении графа , каждый **отдельный** пустой узел в графе задается **отдельным** идентификатором пустого узла
- 2) В отличие от IRI и литералов , идентификаторы пустых узлов не считаются фактическими частями графа RDF
- 3) Идентификаторы пустых узлов являются лишь способом представления пустых узлов в графе (и отличия одного пустого узла от другого) , когда граф записан в триплетном виде
- 4) Идентификаторы пустых узлов имеют значение только в пределах троек , представляющих один граф (два разных графа с тем же количеством пустых узлов могут самостоятельно использовать одни и те же идентификаторы пустых узлов, и было бы неправильно полагать, что пустые узлы из разных графов , имеющие одни и те же идентификаторы пустых узлов одинаковы) .
- 5) Если предполагается , что на пустой узел в графе будет необходимо ссылаться извне графа, для его идентификации должны быть назначены **URI** .

Пустые узлы вместо URIs

Пустые узлы предоставляют возможность делать более точные высказывания о ресурсах , которые могут не иметь URI , но описаны в терминах отношений с другими ресурсами, которые действительно имеют URI.

- Например:

1) У человека есть email address, но нет URI, <mailto:jane@example.org>.

2) Organization has Web page URL, <http://www.example.com/>

Корректно ли использовать URLs в качестве URIs для объектов реального мира???

Пустые узлы вместо URIs

Когда сама Джейн не имеет URI, пустой узел обеспечивает более точный способ моделирования этой ситуации.

Джейн может быть представлена пустым узлом, и этот пустой узел используется в качестве субъекта высказывания с `externs:mailbox` в качестве свойства и URI `mailto:jane@example.org` в качестве его значения.

Пустой узел также может быть описан свойством `rdf:type`, имеющим значение `externs:Person`, и свойством `externs:name`, имеющее значение `"Jane Smith"`, и любой другой описательной информацией, которая может быть полезна, как показано в следующих тройках:

```
_:jane externs:mailbox <mailto:jane@example.org> .
```

```
_:jane rdf:type      externs:Person .
```

```
_:jane externs:name   "Jane Smith" .
```

```
_:jane externs:emplID "23748" .
```

```
_:jane externs:age    "26" .
```

Пустые узлы вместо литералов

В описании книги Джейн , не имея URI для идентификации автора , публикатор данных мог бы написать (используя собственный словарь **ex2terms**:

```
ex2terms:book78354  rdf:type          ex2terms:Book .
```

```
ex2terms:book78354  ex2terms:author  "Jane Smith" .
```

Это правильно???

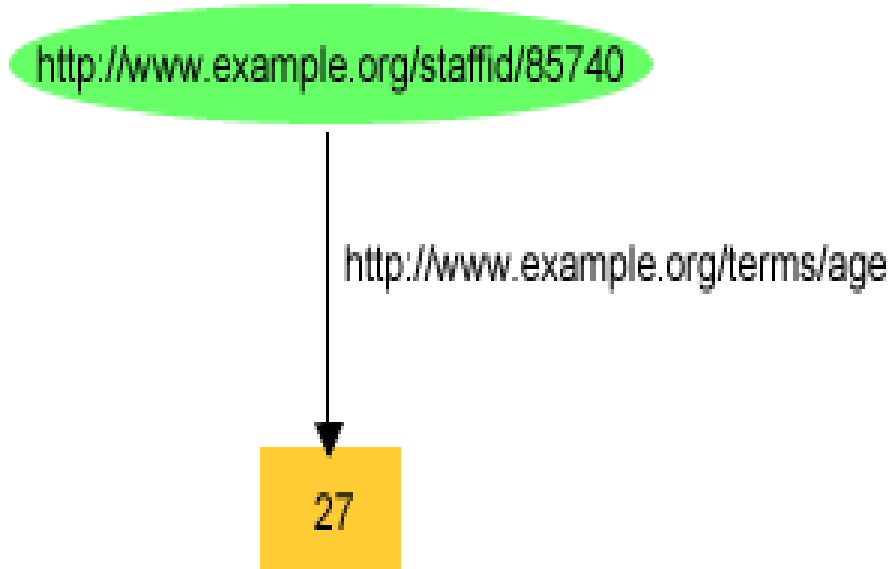
Пустые вершины вместо литералов

Эту же информацию можно записать более корректно при помощи пустых вершин

```
ex2terms:book78354 rdf:type      ex2terms:Book .
ex2terms:book78354 ex2terms:author _:author78354 .
_:author78354      rdf:type      ex2terms:Person .
_:author78354      ex2terms:name  "Jane Smith" .
```

Смысл: «ресурс `ex2terms:book78354` имеет тип `ex2terms:Book`, а его автор это ресурс типа `ex2terms:Person`, его зовут `Jane Smith`."

Типизированные литералы



Обычная практика в языках программирования или системах управления базами данных является предоставление дополнительной информации о том, как интерпретировать литерал, связывая с литералом тип данных , в данном случае, десятичное или целое число.

В RDF, для обеспечения такого рода информации используются **типизированные литералы** ([typed literals](#))

Типизированные литералы

Типизированный литерал RDF = строка с URI, который идентифицирует конкретный тип данных.

Значение, представленное типизированным литералом, - это значение, которое указанный тип данных ассоциирует с указанной строкой.

Example:

<http://www.example.org/staffid/85740>

<<http://www.example.org/terms/age>>

"27"^^<<http://www.w3.org/2001/XMLSchema#integer>> .

Или используя сокращение :

exstaff:85740 exterms:age "27"^^xsd:integer .

Типизированные литералы (xsd:integer) .

<http://www.example.org/staffid/85740>

<http://www.example.org/terms/age>

27

<http://www.example.org/staffid/85740>

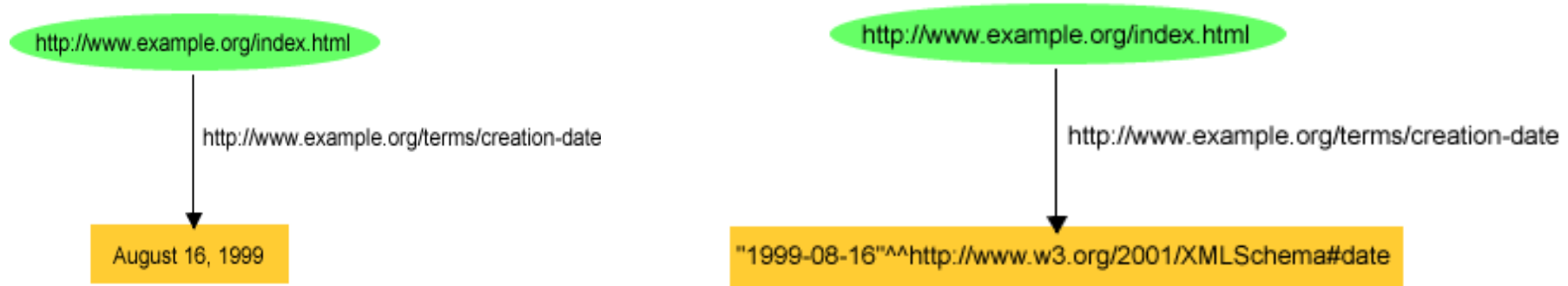
<http://www.example.org/terms/age>

"27"^^<http://www.w3.org/2001/XMLSchema#integer>

- RDF не имеет встроенного набора собственных типов данных.
- Типы данных, используемые в типизированных литералах определены **внешним образом** к RDF, и задаются при помощи URI типов данных.

Еще пример типизированного литерала (`xsd:date`).

ex:index.html exterm:s:creation-date "1999-08-16"^^xsd:date .



Недействительный типизированный литерал

exstaff:85740 exterms:age "pumpkin"^^xsd:integer .

<http://www.example.org/staffid/85740>

<http://www.example.org/terms/age>

"pumpkin"^^<http://www.w3.org/2001/XMLSchema#integer>

Сериализации RDF: RDF/XML и Turtle

- Концептуальная модель в RDF представляет собой граф.
- RDF обеспечивает синтаксис XML для записывания и обмена RDF графами, называемый RDF/XML.
- В отличие от троек, которые были предназначены для сокращенной записи, RDF/XML изначально являлся нормативным синтаксисом для написания RDF.
- RDF/XML определяется в Спецификации Синтаксиса RDF/XML
- Turtle первоначально был языком для запросов SPARQL

What is an XML Element?

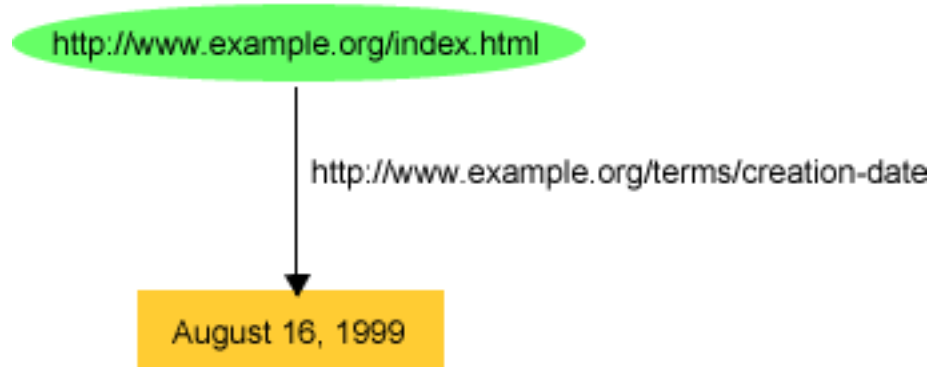
```
<bookstore>
  <book category="CHILDREN">
    <title>Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year>
    <price>29.99</price>
  </book>
  <book category="WEB">
    <title>Learning XML</title>
    <author>Erik T. Ray</author>
    <year>2003</year>
    <price>39.95</price>
  </book>
</bookstore>
```

Элементом XML является все, от (включая) открывающего тега элемента до (включая) конечного тега элемента.

Элемент может содержать:

- другие элементы
- текст
- атрибуты
- или сочетание всего вышеперечисленного ...

Одно высказывание RDF/XML

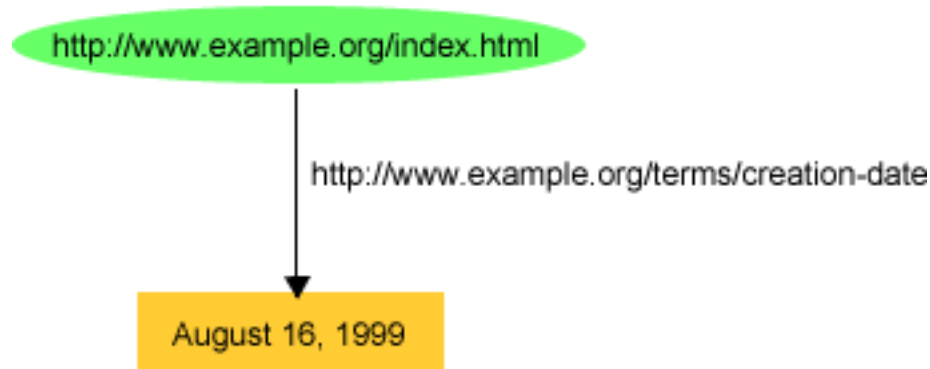


Представление тройки:
ex:index.html
ex:terms:creation-date
"August 16, 1999" .

Add XML declaration:

1. `<?xml version="1.0"?>`

Одно высказывание: RDF/XML

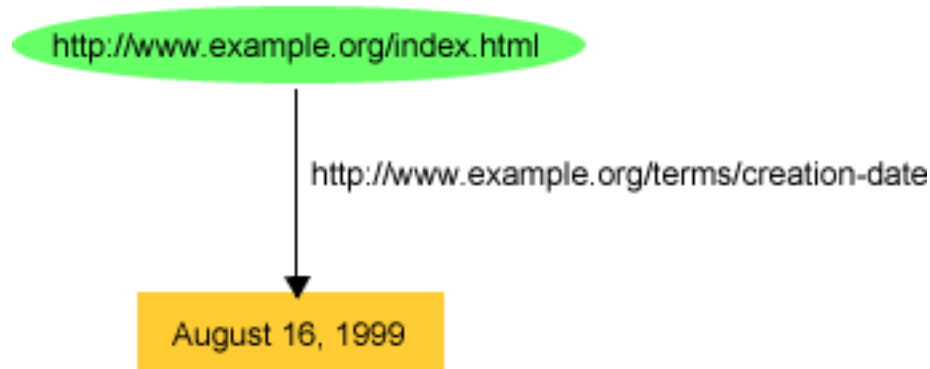


Представление тройки:
ex:index.html
ex:terms:creation-date
"August 16, 1999" .

Добавить корневую вершину RDF

1. `<?xml version="1.0"?>`
2. `<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#" ...`
3. `...`
7. `</rdf:RDF>`

Одно высказывание: RDF/XML



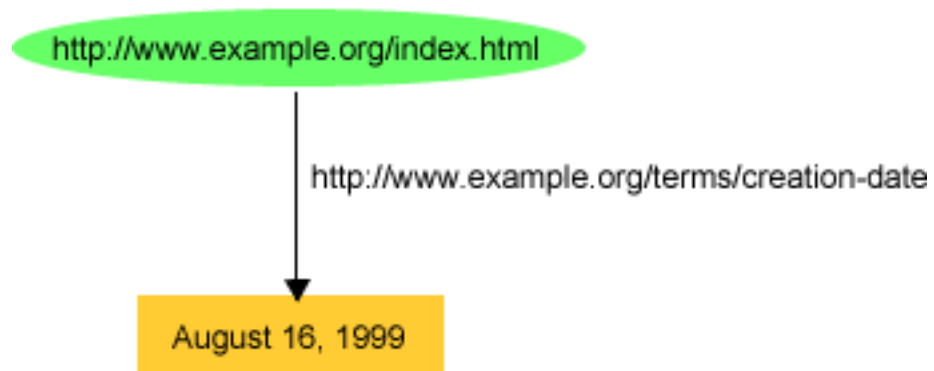
Представление тройки:
ex:index.html
ex:terms:creation-date
"August 16, 1999" .

Добавить пространства имен:

RDF/XML

```
1. <?xml version="1.0"?>
2. <rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-
rdf-syntax-ns#"
3.     xmlns:ex="http://www.example.org/terms/">
.   .   .   .   .
7. </rdf:RDF>
```

Одно высказывание :RDF/XML



Представление тройки :

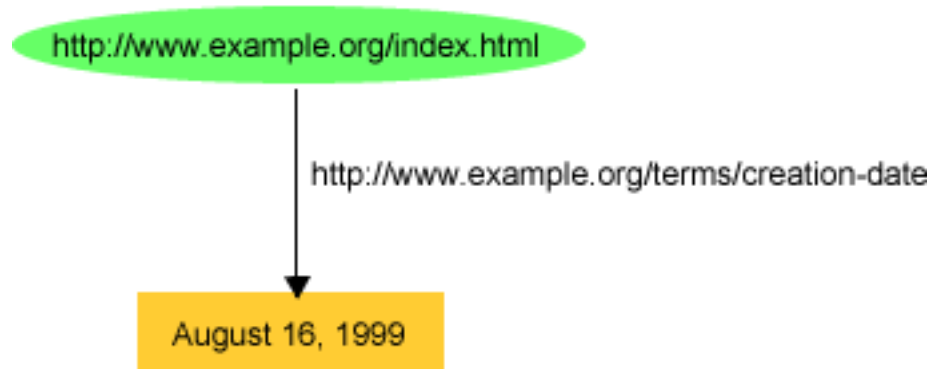
`ex:index.html`

`ex:terms:creation-date` "August 16, 1999" .

Добавить высказывание:

```
1. <?xml version="1.0"?>
2. <rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
3.           xmlns:ex="http://www.example.org/terms/">
4.   <rdf:Description
rdf:about="http://www.example.org/index.html">
5.     .....
6.   </rdf:Description>
7. </rdf:RDF>
```

Одно высказывание: RDF/XML



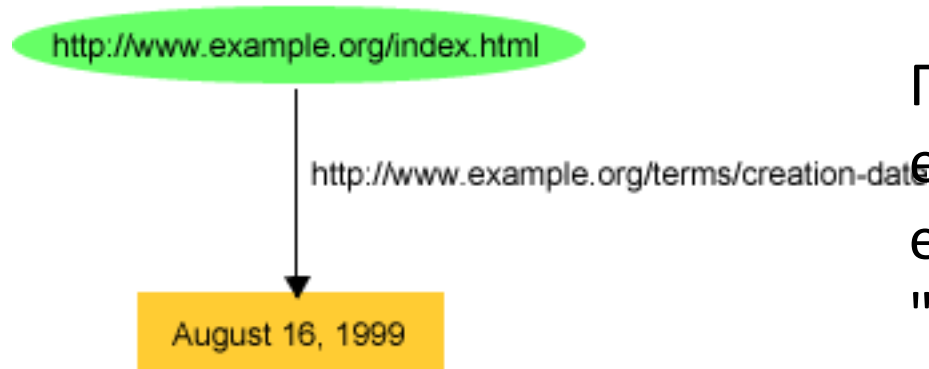
Представление тройки :
ex:index.html
ex:terms:creation-date
"August 16, 1999"

Add Predicates

RDF/XML for the Web Page's Creation Date

1. `<?xml version="1.0"?>`
2. `<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">`
3. `xmlns:ex="http://www.example.org/terms/">`
4. `<rdf:Description`
`rdf:about="http://www.example.org/index.html">`
5. `<ex:creation-date>August 16,`
`1999</ex:creation-date>` (это называется «элемент свойства»)
6. `</rdf:Description>`
7. `</rdf:RDF>`

Одно высказывание: RDF/XML



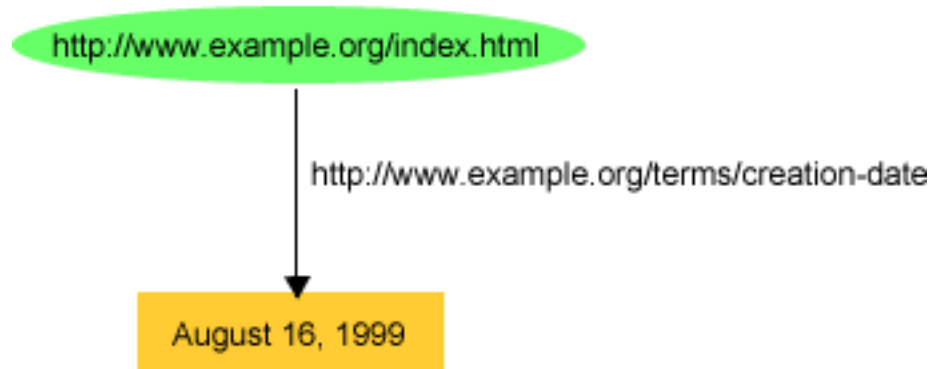
Представление тройки :
`ex:index.html`
`ex:terms:creation-date`
`"August 16, 1999"` .

Добавить конечный тег: `</rdf:RDF>`

RDF/XML for the Web Page's Creation Date

1. `<?xml version="1.0"?>`
2. `<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#" xmlns:ex="http://www.example.org/terms/">`
3. `<rdf:Description rdf:about="http://www.example.org/index.html">`
4. `<ex:creation-date>August 16, 1999</ex:creation-date>`
5. `</rdf:Description>`
6. `</rdf:RDF>`

Одно высказывание :Turtle



Представление тройки:
ex:index.html
exterm:s:creation-date
"August 16, 1999" .

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.
@prefix exterm:s: <http://www.example.org/terms/>.

<http://www.example.org/index.html> exterm:s:creation-date "August 16, 1999".

В чем разница с представлением в виде тройки?

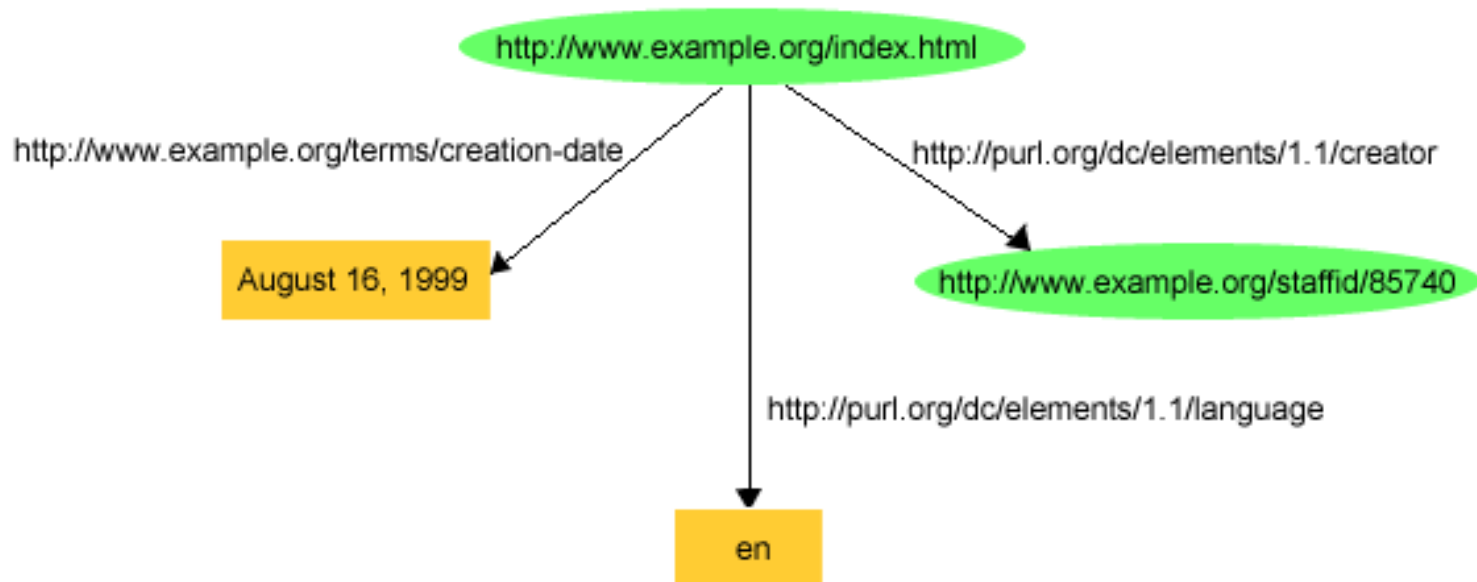
Несколько предикатов

Пример группы высказываний про `http://www.example.org/index.html`:

`ex:index.html` `dc:creator` [exstaff:85740](http://www.example.org/staffid/85740) .

`ex:index.html` `exterm:creation-date` "August 16, 1999" .

`ex:index.html` `dc:language` "en" .



Несколько предикатов: RDF/XML

1. <?xml version="1.0"?>
2. <rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
3. xmlns:dc="http://purl.org/dc/elements/1.1/"
4. xmlns:exterms="http://www.example.org/terms/">
5. <rdf:Description rdf:about="http://www.example.org/index.html">
6. <exterms:creation-date>August 16, 1999</exterms:creation-date>
7. </rdf:Description>

8. <rdf:Description rdf:about="http://www.example.org/index.html">
9. <dc:language>en</dc:language>
10. </rdf:Description>

11. <rdf:Description rdf:about="http://www.example.org/index.html">
12. <dc:creator rdf:resource="http://www.example.org/staffid/85740"/>
13. </rdf:Description>
14. </rdf:RDF>

Сокращение для нескольких свойств:

RDF/XML

1. `<?xml version="1.0"?>`
2. `<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#" xmlns:dc="http://purl.org/dc/elements/1.1/" xmlns:exterms="http://www.example.org/terms/">`
3. `xmlns:dc="http://purl.org/dc/elements/1.1/"`
4. `xmlns:exterms="http://www.example.org/terms/">`
5. `<rdf:Description rdf:about="http://www.example.org/index.html">`
6. `<exterms:creation-date>August 16, 1999</exterms:creation-date>`
7. `<dc:language>en</dc:language>.`
8. `<dc:creator`
9. `rdf:resource="http://www.example.org/staffid/85740"/>`
10. `</rdf:Description>`
11. `</rdf:RDF>`

Несколько предикатов :Turtle

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.

@prefix dc: <http://purl.org/dc/elements/1.1/#>.

@prefix extermis: <http://www.example.org/terms/>.

<http://www.example.org/index.html> extermis:creation-date
"August 16, 1999".

<http://www.example.org/index.html> dc:language "en".

<http://www.example.org/index.html> dc:creator
<http://www.example.org/staffid/85740>.

- **Предикаты в сокращенном виде, ресурсы - полностью**

Сокращение для нескольких свойств : Turtle

(;)

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.

@prefix dc: <http://purl.org/dc/elements/1.1/#>.

@prefix exterm: <http://www.example.org/terms/>.

<http://www.example.org/index.html>

exterm:creation-date "August 16, 1999";

dc:language "en";

dc:creator <http://www.example.org/staffid/85740>.

Сокращение для нескольких свойств : Turtle (,)

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.

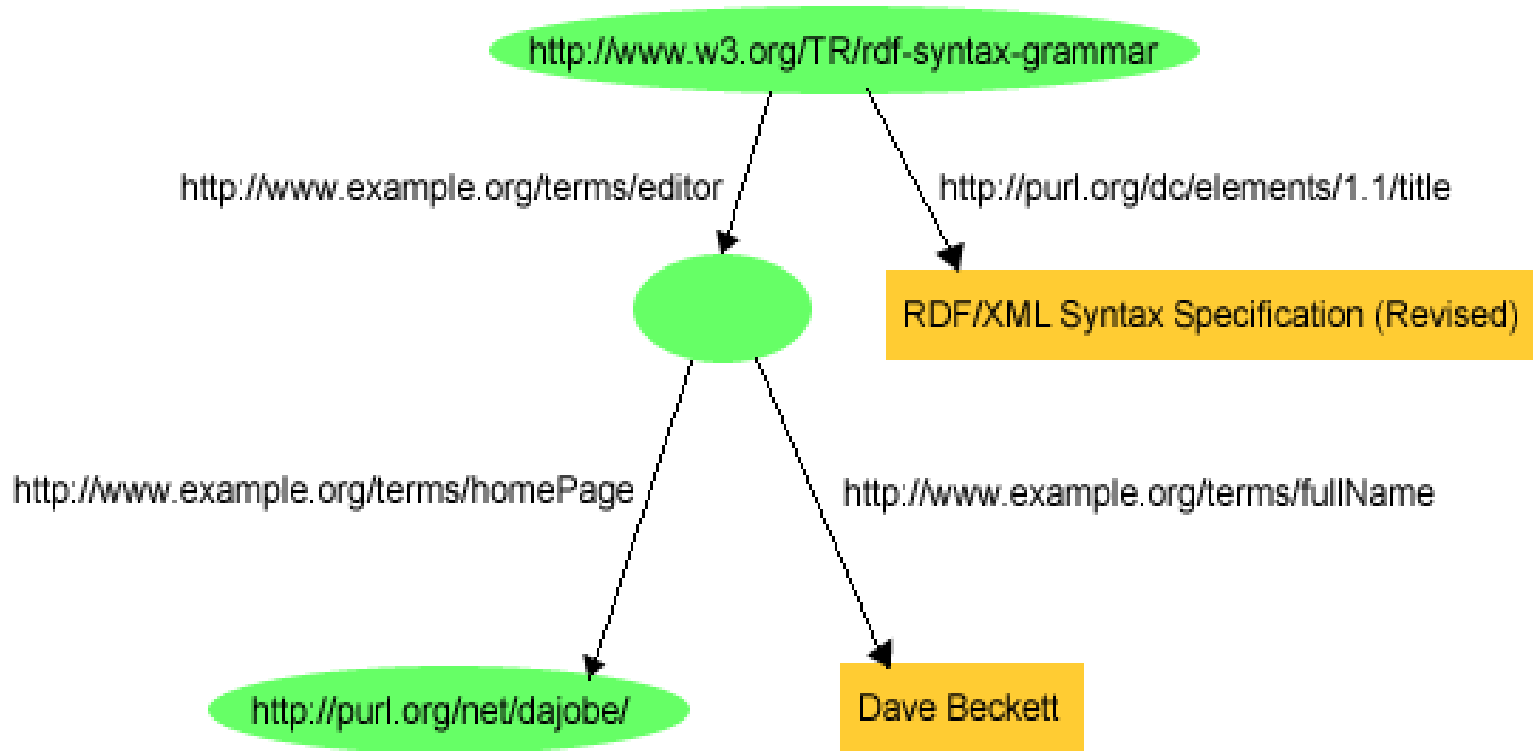
@prefix dc: <http://purl.org/dc/elements/1.1/#>.

@prefix exterms: <http://www.example.org/terms/>.

<http://www.example.org/index.html>

dc:creator <http://www.example.org/staffid/85740,
<http://www.example.org/staffid/85741>.

Граф с пустым узлом



Пустой узел используется как Object в первом высказывании и как Subject во втором высказывании

RDF/XML, описывающий пустой узел

1. `<?xml version="1.0"?>`
2. `<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
3. xmlns:dc="http://purl.org/dc/elements/1.1/"
4. xmlns:exterms="http://example.org/stuff/1.0/">`
5. `<rdf:Description rdf:about="http://www.w3.org/TR/rdf-syntax-grammar">`
6. `<dc:title>RDF/XML Syntax Specification (Revised)</dc:title>`
7. `<exterms:editor rdf:nodeID="abc"/>`
8. `</rdf:Description>`
9. `<rdf:Description rdf:nodeID="abc">`
10. `<exterms:fullName>Dave Beckett</exterms:fullName>`
11. `<exterms:homePage rdf:resource="http://purl.org/net/dajobe/" />`
12. `</rdf:Description>`
13. `</rdf:RDF>`

Turtle, описывающий пустой узел (1)

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.

@prefix dc: <http://purl.org/dc/elements/1.1/#>.

@prefix exterm: <http://www.example.org/terms/>.

<http://www.w3.org/TR/rdf-syntax-grammar>

dc:title "RDF/XML Syntax Specification (Revised)";

exterm:editor _:abc.

_:abc

exterm:fullName "Dave Beckett";

exterm:homePage <http://purl.org/net/dajobe/>.

Turtle, описывающий пустой узел (2)

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.

@prefix dc: <http://purl.org/dc/elements/1.1/#>.

@prefix exterm: <http://www.example.org/terms/>.

<http://www.w3.org/TR/rdf-syntax-grammar>

dc:title "RDF/XML Syntax Specification (Revised)";

exterm:editor [

exterm:fullName "Dave Beckett";

exterm:homePage <http://purl.org/net/dajobe/>

].

Использование типизированного литерала в RDF/XML

Типизированный элемент представляется в RDF/XML добавлением атрибута `rdf:datatype` указывающим на URI типа данных к элементу свойства, содержащему литерал.

For example,

ex:index.html exterm:s:creation-date "1999-08-16"^^xsd:date .

1. `<?xml version="1.0"?>`
2. `<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#" xmlns:exterm:s="http://www.example.org/terms/">`
3. `xmlns:exterm:s="http://www.example.org/terms/">`
4. `<rdf:Description rdf:about="http://www.example.org/index.html">`
5. `<exterm:s:creation-date rdf:datatype="http://www.w3.org/2001/XMLSchema#date">1999-08-16`
 `</exterm:s:creation-date>`
6. `</rdf:Description>`
7. `</rdf:RDF>`

RDF/XML Использование типизированного литерала и XML Entity

- 1. `<?xml version="1.0"?>`
- 2. `<!DOCTYPE rdf:RDF [<!ENTITY xsd`
`"http://www.w3.org/2001/XMLSchema#">]>`
-
- 3. `<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"`
- 4. `xmlns:exterms="http://www.example.org/terms/">`
-
- 5. `<rdf:Description rdf:about="http://www.example.org/index.html">`
- 6. `<exterms:creation-date rdf:datatype="&xsd:date">1999-08-16`
- `</exterms:creation-date>`
- 7. `</rdf:Description>`
-
- 8. `</rdf:RDF>`

Граф RDF записывается в RDF/XML следующим образом:

- 1) Всем пустым узлам назначаются идентификаторы пустых узлов.
- 2) Каждый узел указывается по очереди в качестве субъекта не вложенного элемента `rdf:Description`,
 - a) с помощью атрибута `rdf:about` , если узел имеет `URI`,
 - b) или атрибута `rdf:nodeID`, если узел является `пустым`.
- 3) Для каждого узла, указанного в качестве субъекта, в свою очередь создается соответствующий элемент свойства с:
 - a) литеральным контентом(возможно пустым)
 - b) атрибутом `rdf:resource` , указывающим объект тройки (если объектный узел имеет `URI`),
 - c) атрибутом `rdf:nodeID` , указывающим объект тройки (если объект является пустым узлом).

Сокращение и организация RDF URI (RDF/XML)

RDF/XML for example.com's Catalog

1. `<?xml version="1.0"?>`
2. `<!DOCTYPE rdf:RDF [<!ENTITY xsd "http://www.w3.org/2001/XMLSchema#">]>`
3. `<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">`
4. `xmlns:exterms="http://www.example.com/terms/">`
5. `<rdf:Description rdf:ID="item10245">`
6. `<exterms:model rdf:datatype="&xsd:string">Overnighter</exterms:model>`
7. `<exterms:sleeps rdf:datatype="&xsd:integer">2</exterms:sleeps>`
8. `<exterms:weight rdf:datatype="&xsd:decimal">2.4</exterms:weight>`
9. `<exterms:packedSize rdf:datatype="&xsd:integer">784</exterms:packedSize>`
10. `</rdf:Description>`

...описание других продуктов...

11. `</rdf:RDF>`

URIref [http:// www.example.com/2002/04/products #
item10245](http://www.example.com/2002/04/products#item10245)

Сокращение и организация RDF URI (RDF/XML)

1) Любое другое высказывание RDF/XML в этом каталоге может ссылаться на палатку либо с помощью :

а) абсолютного URI

<http://www.example.com/2002/04/products#item10245>,

б) относительного *URI* [#item10245](#).

2) RDF высказывание **вне** каталога должно ссылаться на палатку **только** с помощью (полного) **абсолютного** URI:

<http://www.example.com/2002/04/products#item10245>.

Сокращение и организация RDF URI (RDF/XML)

RDF/XML for example.com's Catalog

1. `<?xml version="1.0"?>`
2. `<!DOCTYPE rdf:RDF [<!ENTITY xsd "http://www.w3.org/2001/XMLSchema#">]>`
3. `<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">`
4. `xmlns:exterms="http://www.example.com/terms/">`
5. `<rdf:Description rdf:about="#item10245">`
6. `<exterms:model rdf:datatype="&xsd:string">Overnighter</exterms:model>`
7. `<exterms:sleeps rdf:datatype="&xsd:integer">2</exterms:sleeps>`
8. `<exterms:weight rdf:datatype="&xsd:decimal">2.4</exterms:weight>`
9. `<exterms:packedSize rdf:datatype="&xsd:integer">784</exterms:packedSize>`
10. `</rdf:Description>`

...описания других продуктов...

11. `</rdf:RDF>`

URIref [http:// www.example.com/2002/04/products #
item10245](http://www.example.com/2002/04/products#item10245)

Сокращение и организация RDF URI (Turtle)

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.

@prefix xsd: <http://www.w3.org/2001/XMLSchema#>.

@prefix exterm: <http://www.example.org/terms/>.

[:item10245](#)

exterm:models "Overnighter"^^xsd:string;

exterm:sleeps "2"^^xsd:integer;

exterm:weight "2.4"^^xsd:decimal;

exterm:packedSize "784"^^xsd:integer.

... описания других продуктов...

Использование имен без префикса определяет **идентификатор фрагмента**,
как сокращение полного URIref описываемого ресурса.

Full URIref [http:// www.example.com/2002/04/products #
item10245](http://www.example.com/2002/04/products#item10245)

Сокращение и организация RDF URIs

Любой другой RDF/XML (Turtle) **внутри** этого каталога может ссылаться на терм, используя либо абсолютный URIref

<http://www.example.com/2002/04/products#item10245>,

Либо относительный URIref
[#item10245](#).

Относительный URIref , будет пониматься как URIref **определенный относительно базового URIref** каталога.

Пример внешней ссылки (RDF/XML)

1. `<?xml version="1.0"?>`
2. `<!DOCTYPE rdf:RDF [<!ENTITY xsd`
 `"http://www.w3.org/2001/XMLSchema#">]>`
3. `<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">`
4. `xmlns:sportex="http://www.exampleRatings.com/terms/">`
5. `<rdf:Description`
 `rdf:about="http://www.example.com/2002/04/products#item10245">`
6. `<sportex:ratingBy rdf:datatype="&xsd:string">Richard`
 `Roe</sportex:ratingBy>`
7. `<sportex:numberStars`
 `rdf:datatype="&xsd:integer">5</sportex:numberStars>`
8. `</rdf:Description>`
9. `</rdf:RDF>`

Пример внешней ссылки(Turtle)

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.

@prefix sportex: <http://www.exampleRatings.com/terms/>.

@prefix xsd: <http://www.w3.org/2001/XMLSchema#>.

<<http://www.example.com/2002/04/products#item10245>>

sportex:ratingBy "Richard Roe"^^xsd:string;

sportex:numberStars "5"^^xsd:integer.

Эти примеры также иллюстрируют:

- 1) Одним из основных архитектурных принципов Web является то, что кто угодно должен иметь возможность свободно добавлять информацию о существующих ресурсах , используя любой словарь какой ему заблагорассудится [\[BERNERS-LEE98\]](#).
- 2) RDF, описывающий определенный ресурс, не обязан быть расположенным в одном месте , вместо этого файлы могут быть распределены по всей сети.

Это верно для ситуаций, когда:

- a) одна организация оценивает или комментирует ресурс , определенный другой организацией,
- b) исходный автор, определивший ресурс (или кто-либо еще) желает усилить описание этого ресурса , предоставляя дополнительную информацию о нем.

Этого можно достигнуть:

- a) Модифицируя документ RDF в котором ресурс был первоначально описан, добавляя новые свойства и значения.
- b) Создав отдельный документ, добавляющий новые свойства и значения к исходному ресурсу через его URIref.

Использование Base в example.com's Catalog

Приведенное выше обсуждение показало, что относительные URIs, такие как [#item10245](#) будут интерпретироваться по отношению к **базовому URI**.

По умолчанию, этим **базовым URI** будет URI ресурса, в котором **используется** относительная URIref .

Тем не менее, в некоторых случаях желательно иметь возможность **явно** указать этот базовый URI.

Например, предположим, что в дополнение к каталогу , расположенному по адресу <http://www.example.com/2002/04/products>,

example.org захочет разместить дубликат каталога на зеркальном сайте <http://mirror.example.com/2002/04/products>.

Это может создать проблему , так как если каталог был доступен с сайта-зеркала, URIref для палатки будет генерироваться из URI содержащего документа , образуя <http://mirror.example.com/2002/04/products> # item10245 , а не <http://www.example.com/2002/04/products> # item10245. Значит, базу надо указать **ЯВНО**.

Кроме того, example.org может назначить базовую URIref для своего набора продукции, вообще не публикуя один исходный документ расположение которого определяет URIref.

Using XML Base (RDF/XML)

1. `<?xml version="1.0"?>`
2. `<!DOCTYPE rdf:RDF [<!ENTITY xsd "http://www.w3.org/2001/XMLSchema#">]>`
3. `<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">`
4. `xmlns:exterms="http://www.example.com/terms/"`
5. `xml:base="http://www.example.com/2002/04/products">`
6. `<rdf:Description rdf:ID="item10245">`
7. `<exterms:model rdf:datatype="&xsd:string">Overnighter</exterms:model>`
8. `<exterms:sleeps rdf:datatype="&xsd:integer">2</exterms:sleeps>`
9. `<exterms:weight rdf:datatype="&xsd:decimal">2.4</exterms:weight>`
10. `<exterms:packedSize rdf:datatype="&xsd:integer">784</exterms:packedSize>`
11. `</rdf:Description>`

...описание других продуктов...

12. `</rdf:RDF>`

Использование Base(Turtle)

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.

@prefix exterm: <http://www.example.org/terms/>.

@prefix xsd: <http://www.w3.org/2001/XMLSchema#>.

@base <http://www.example.com/2002/04/products>.

:item10245

exterm:models "Overnighter"^^xsd:string;

exterm:sleeps "2"^^xsd:integer;

exterm:weight "2.4"^^xsd:decimal;

exterm:packedSize "784"^^xsd:integer.

... описание других продуктов...

Описание класса Tent при помощи `rdf:type` (RDF/XML)

1. `<?xml version="1.0"?>`
2. `<!DOCTYPE rdf:RDF [<!ENTITY xsd "http://www.w3.org/2001/XMLSchema#">]>`
3. `<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">`
4. `xmlns:exterms="http://www.example.com/terms/"`
5. `xml:base="http://www.example.com/2002/04/products">`
6. `<rdf:Description rdf:ID="item10245">`
7. `<rdf:type rdf:resource="http://www.example.com/terms/Tent"/>`
8. `<exterms:model rdf:datatype="&xsd:string">Overnighter</exterms:model>`
9. `<exterms:sleeps rdf:datatype="&xsd:integer">2</exterms:sleeps>`
10. `<exterms:weight rdf:datatype="&xsd:decimal">2.4</exterms:weight>`
11. `<exterms:packedSize`
 `rdf:datatype="&xsd:integer">784</exterms:packedSize>`
12. `</rdf:Description>`
- ... описание других продуктов...
13. `</rdf:RDF>`

Сокращение Tent's Type (RDF/XML)

типизированные элементы Node

1. `<?xml version="1.0"?>`
2. `<!DOCTYPE rdf:RDF [<!ENTITY xsd "http://www.w3.org/2001/XMLSchema#">]>`
3. `<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">`
4. `xmlns:exterms="http://www.example.com/terms/"`
5. `xml:base="http://www.example.com/2002/04/products">`
6. `<exterms:Tent rdf:ID="item10245">`
7. `<exterms:model rdf:datatype="&xsd:string">Overnighter</exterms:model>`
8. `<exterms:sleeps rdf:datatype="&xsd:integer">2</exterms:sleeps>`
9. `<exterms:weight rdf:datatype="&xsd:decimal">2.4</exterms:weight>`
10. `<exterms:packedSize rdf:datatype="&xsd:integer">784</exterms:packedSize>`
11. `</exterms:Tent>`

... описание других продуктов...

12. `</rdf:RDF>`

Описание класса Tent при помощи rdf:type (Turtle)

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.

@prefix exterm: <http://www.example.org/terms/>.

@prefix xsd: <http://www.w3.org/2001/XMLSchema#>.

@base <http://www.example.com/2002/04/products>.

:item10245

 rdf:type <http://www.example.com/terms/Tent>;

 exterm:models "Overnighter"^^xsd:string;

 exterm:sleeps "2"^^xsd:integer;

 exterm:weight "2.4"^^xsd:decimal;

 exterm:packedSize "784"^^xsd:integer.

Сокращение the Tent's Type

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.

@prefix exterm: <http://www.example.org/terms/>.

@prefix xsd: <http://www.w3.org/2001/XMLSchema#>.

@base <http://www.example.com/2002/04/products>.

:item10245 a <http://www.example.com/terms/Tent>;

 exterm:models "Overnighter"^^xsd:string;

 exterm:sleeps "2"^^xsd:integer;

 exterm:weight "2.4"^^xsd:decimal;

 exterm:packedSize "784"^^xsd:integer.

RDF=>RDFS, OWL...

- Сам RDF не предоставляет средства для определения классов вещей для конкретных приложений , таких как палатка в этом примере, или их свойств, таких как *exterms:weight*.
- Вместо этого, такие классы будут описаны в схеме RDF, используя язык *RDF schema*.
- Другие такие средства для описания классов также могут быть определены, например, *DAML+OIL* и *OWL*.